



دانشکده علوم ریاضی
دانشگاه فردوسی مشهد

پروژه دوم درس طراحی الگوریتم

بازسازی شبکه مخابراتی پس از طوفان مغناطیسی

استاد درس: دکتر رضا قنبری

اعضای گروه:

سارا مسگر مقدم، اسما موحدی تبار

فهرست مطالب

۲	۱ فاز چهارم: بازسازی شبکه با کمینه‌سازی هزینه ساخت تحت قید بودجه نگهداری
۳	۱.۱ عدم کفایت الگوریتم‌های حریصانه
۵	۲.۱ روش حل
۵	۳.۱ مرتب‌سازی اولیه کابل‌ها
۵	۴.۱ ساختار گره‌های درخت جستجو
۶	۵.۱ روش انشعاب
۷	۶.۱ بررسی کابل‌های سالم
۷	۷.۱ جلوگیری از تشکیل چرخه
۸	۸.۱ تابع کران پایین
۹	۹.۱ صحت تابع کران پایین
۱۰	۱۰.۱ شرایط هرس
۱۱	۱۱.۱ تشخیص جواب نهایی
۱۲	۱۲.۱ پیچیدگی زمانی
۱۲	۱۳.۱ جمع‌بندی

۱ فاز چهارم: بازسازی شبکه با کمینه‌سازی هزینه ساخت تحت قید بودجه نگهداری

در این فاز، مسئله بازسازی شبکه نسبت به فازهای پیشین توسعه یافته است. برای هر کابل پیشنهادی علاوه بر هزینه ساخت اولیه، یک هزینه نگهداری سالیانه نیز در نظر گرفته می‌شود. بنابراین هر کابل جدید با دو پارامتر مستقل مشخص می‌شود:

• هزینه ساخت اولیه (w_i)

• هزینه نگهداری سالیانه (m_i)

علاوه بر این، بخشی از کابل‌های موجود پس از وقوع طوفان سالم باقی مانده‌اند و می‌توان از آن‌ها بدون پرداخت هزینه ساخت استفاده کرد. مطابق صورت مسئله، هزینه نگهداری این کابل‌ها در بودجه تخصیص یافته برای کابل‌های جدید محاسبه نمی‌شود؛ بنابراین محدودیت بودجه تنها به کابل‌های جدید اعمال می‌شود.

هدف، یافتن یک درخت پوشای شبکه است که به طور هم‌زمان شرایط زیر را ارضا کند:

۱. تمامی شهرها را به یکدیگر متصل نماید.

۲. مجموع هزینه ساخت کابل‌های جدید حداقل باشد.

۳. مجموع هزینه نگهداری کابل‌های جدید از بودجه تعیین شده B تجاوز نکند.

به بیان دیگر، اگر مجموعه کابل‌های انتخاب شده را با T نمایش دهیم، مسئله به صورت زیر مدل می‌شود:

$$\min \sum_{e_i \in T} w_i$$

به طوری که

$$\sum_{e_i \in T} m_i \leq B$$

و مجموعه یال‌های انتخاب شده یک درخت پوشا را تشکیل دهد.

این مسئله در ادبیات طراحی الگوریتم با عنوان Constrained Minimum Spanning Tree (CMST) شناخته می‌شود؛ زیرا علاوه بر کمینه‌سازی تابع هدف، یک قید اضافی نیز باید هم‌زمان ارضا شود. وجود این قید سبب

می‌شود بسیاری از ویژگی‌های مسئله کلاسیک درخت پوشای کمینه از جمله خاصیت انتخاب حریصانه دیگر برقرار نباشد.

۱.۱ عدم کفایت الگوریتم‌های حریصانه

در مسئله کلاسیک درخت پوشای کمینه، الگوریتم‌های حریصانه‌ای مانند الگوریتم کراسکال و پریم همواره جواب بهینه را تولید می‌کنند، زیرا تنها معیار تصمیم‌گیری، کمینه بودن مجموع هزینه ساخت یال‌ها است. با این حال، در مسئله حاضر علاوه بر هزینه ساخت، محدودیت مستقلی نیز بر مجموع هزینه نگهداری کابل‌های جدید اعمال شده است.

از آنجا که الگوریتم‌های حریصانه تنها بر اساس هزینه ساخت تصمیم‌گیری می‌کنند، ممکن است در مراحل ابتدایی کابل‌هایی با هزینه ساخت اندک اما هزینه نگهداری بسیار زیاد انتخاب شوند. در نتیجه، اگرچه هزینه ساخت کاهش می‌یابد، اما بودجه نگهداری زودتر از موعد مصرف شده و امکان تکمیل درخت پوشا با رعایت محدودیت بودجه از بین می‌رود. بنابراین انتخاب محلی بهینه، لزوماً به جواب سراسری بهینه منجر نخواهد شد و خاصیت انتخاب حریصانه از بین می‌رود.

در نتیجه، الگوریتم‌های کلاسیک مانند کراسکال و پریم دیگر تضمینی برای یافتن جواب بهینه این مسئله ارائه نمی‌کنند.

مثال نقض

فرض کنید گراف شامل سه رأس و سه یال مطابق جدول زیر باشد.

یال	هزینه ساخت	هزینه نگهداری
(1, 2)	1	9
(2, 3)	1	9
(1, 3)	4	1

فرض کنید بودجه نگهداری برابر باشد با

$$B = 10.$$

الگوریتم کراسکال مطابق روال معمول، ابتدا دو یال با کمترین هزینه ساخت، یعنی

$$(1, 2), \quad (2, 3)$$

را انتخاب می‌کند.

در این حالت خواهیم داشت

$$Build = 2,$$

اما مجموع هزینه نگهداری برابر است با

$$Maintenance = 18 > B,$$

در نتیجه این جواب قید مسئله را نقض کرده و قابل قبول نیست.

در مقابل، اگر یال‌های

$$(1, 2), \quad (1, 3)$$

انتخاب شوند، خواهیم داشت

$$Build = 5,$$

و

$$Maintenance = 10,$$

که ضمن رعایت محدودیت بودجه، یک درخت پوشای معتبر نیز تشکیل می‌دهد.

این مثال نشان می‌دهد که انتخاب یال‌های با کمترین هزینه ساخت، لزوماً بهترین جواب مسئله نیست و وجود محدودیت بودجه نگهداری باعث از بین رفتن خاصیت انتخاب حریصانه می‌شود. بنابراین برای حل این مسئله باید از روش‌هایی استفاده شود که فضای تمامی جواب‌های ممکن را به صورت سیستماتیک بررسی کرده و در عین حال با حذف حالت‌های غیرامیدبخش، زمان اجرا را کاهش دهند.

در این پروژه، برای دستیابی به چنین هدفی از الگوریتم Branch and Bound استفاده شده است.

۲.۱ روش حل

برای حل مسئله از الگوریتم Branch and Bound استفاده شده است. این روش تمامی جواب‌های ممکن را به صورت سیستماتیک بررسی می‌کند، اما با استفاده از یک تابع کران پایین و مجموعه‌ای از شروط هرس، شاخه‌هایی را که امکان رسیدن به جواب بهینه ندارند حذف می‌کند. به این ترتیب، ضمن تضمین بهینگی جواب نهایی، از بررسی بخش بزرگی از فضای حالت جلوگیری می‌شود.

در این الگوریتم، هر گره از درخت جستجو نمایانگر یک جواب جزئی است؛ یعنی مجموعه‌ای از تصمیم‌هایی که تاکنون درباره کابل‌های بررسی شده اتخاذ شده‌اند. با حرکت از ریشه به سمت برگ‌ها، تعداد بیشتری از کابل‌ها تعیین تکلیف شده و در نهایت یک درخت پوشای کامل یا یک شاخه غیرقابل قبول حاصل می‌شود.

۳.۱ مرتب‌سازی اولیه کابل‌ها

پیش از آغاز فرآیند جستجو، تمامی کابل‌ها بر اساس هزینه ساخت به صورت صعودی مرتب می‌شوند. در صورتی که دو کابل هزینه ساخت یکسانی داشته باشند، کابلی که هزینه نگهداری کمتری دارد در اولویت قرار می‌گیرد.

در پیاده‌سازی، این مرتب‌سازی به صورت زیر انجام شده است:

$$(w_1 < m_2) \quad \text{یا در صورت برابری} \quad w_1 = w_2, m_1 < m_2$$

مرتب‌سازی اولیه دو مزیت مهم ایجاد می‌کند.

۱. در محاسبه تابع کران پایین، کابل‌های باقی‌مانده دقیقاً به همان ترتیبی بررسی می‌شوند که الگوریتم کراسکال انتخاب می‌کند؛ بنابراین محاسبه کران پایین ساده‌تر و سریع‌تر انجام می‌شود.

۲. کابل‌های ارزان‌تر در ابتدای درخت جستجو بررسی می‌شوند و معمولاً جواب‌های مناسب در مراحل ابتدایی پیدا می‌شوند. در نتیجه مقدار بهترین جواب موجود (BestCost) زودتر کاهش یافته و هرس شاخه‌های غیرامیدبخش مؤثرتر خواهد شد.

۴.۱ ساختار گره‌های درخت جستجو

هر گره درخت جستجو وضعیت فعلی الگوریتم را نمایش می‌دهد و شامل اطلاعات زیر است.

- اندیس اولین کابل بررسی نشده

- مجموع هزینه ساخت کابل‌های انتخاب‌شده
- مجموع هزینه نگهداری کابل‌های انتخاب‌شده
- تعداد یال‌های انتخاب‌شده تاکنون
- وضعیت ساختار داده (DSU) Disjoint Set Union
- فهرست کابل‌های انتخاب‌شده

بنابراین اگر تاکنون k کابل بررسی شده باشد، گره متناظر دقیقاً بیانگر تمام تصمیم‌های اتخاذ‌شده درباره همین k کابل است و کابل‌های بعدی هنوز تعیین تکلیف نشده‌اند.

استفاده از ساختار DSU این امکان را فراهم می‌کند که در هر گره، وضعیت مؤلفه‌های همبند شبکه نگهداری شده و تشکیل چرخه با هزینه بسیار اندکی بررسی شود.

۵.۱ روش انشعاب

در هر گره، کابلی که اندیس آن برابر با مقدار index است بررسی می‌شود. سپس بسته به نوع کابل، شاخه‌های جدید تولید می‌شوند.

انتخاب کابل جدید

اگر کابل جدید باعث تشکیل چرخه نشود و اضافه شدن آن موجب تجاوز مجموع هزینه نگهداری از بودجه مجاز نشود، یک گره فرزند ایجاد شده و این کابل به جواب فعلی اضافه می‌شود.

در این حالت:

- هزینه ساخت افزایش می‌یابد.
- هزینه نگهداری افزایش می‌یابد.
- تعداد یال‌های انتخاب‌شده یک واحد افزایش می‌یابد.
- ساختار DSU به‌روزرسانی می‌شود.
- اندیس کابل بعدی افزایش پیدا می‌کند.

عدم انتخاب کابل جدید

در شاخه دوم، کابل فعلی کنار گذاشته شده و الگوریتم مستقیماً بررسی کابل بعدی را ادامه می‌دهد. در این شاخه هیچ تغییری در هزینه‌ها یا ساختار DSU ایجاد نمی‌شود و تنها اندیس کابل افزایش می‌یابد. در نتیجه، برای هر کابل جدید معمولاً دو شاخه درخت جستجو ایجاد می‌شود.

۶.۱ بررسی کابل‌های سالم

کابل‌هایی که پس از طوفان سالم باقی مانده‌اند، هزینه ساخت و هزینه نگهداری مؤثر در بودجه ندارند. بنابراین اگر چنین کابلی موجب تشکیل چرخه نشود، انتخاب آن همواره سودمند است؛ زیرا بدون افزایش مقدار تابع هدف، امکان اتصال مؤلفه‌های بیشتری از شبکه را فراهم می‌کند.

به همین دلیل، در پیاده‌سازی تنها یک شاخه برای این کابل‌ها ایجاد می‌شود.

اگر کابل سالم موجب تشکیل چرخه نشود، مستقیماً انتخاب می‌شود؛ در غیر این صورت کنار گذاشته شده و الگوریتم بررسی کابل بعدی را ادامه می‌دهد.

حذف شاخه عدم انتخاب برای کابل‌های سالم باعث کاهش قابل توجه اندازه درخت جستجو و در نتیجه افزایش سرعت اجرای الگوریتم می‌شود، بدون آنکه هیچ جواب بهینه‌ای از دست برود.

۷.۱ جلوگیری از تشکیل چرخه

برای تشخیص تشکیل چرخه از ساختمان داده Disjoint Set Union (DSU) استفاده شده است.

برای هر کابل با دو رأس u و v ابتدا نماینده مجموعه هر دو رأس محاسبه می‌شود.

اگر

$$Find(u) = Find(v)$$

باشد، هر دو رأس در یک مؤلفه قرار دارند و افزودن این کابل موجب تشکیل چرخه خواهد شد؛ بنابراین این کابل قابل انتخاب نیست.

در غیر این صورت، با اجرای عملگر

$$Union(u, v)$$

دو مؤلفه با یکدیگر ادغام شده و کابل به جواب فعلی افزوده می‌شود.

از آنجا که در پیاده‌سازی از فشردگی مسیر (Path Compression) و اجتماع بر اساس رتبه (Union by Rank) استفاده شده است، مرتبه زمانی هر عملیات تقریباً برابر

$$O(\alpha(n))$$

است که در عمل تقریباً ثابت محسوب می‌شود.

ساختار DSU علاوه بر فرآیند انشعاب، در محاسبه تابع کران پایین نیز مورد استفاده قرار می‌گیرد تا بتوان بدون ایجاد چرخه، ارزان‌ترین کابل‌های باقی‌مانده را انتخاب کرد.

۸.۱ تابع کران پایین

کارایی الگوریتم Branch and Bound تا حد زیادی به کیفیت تابع کران پایین (Lower Bound) وابسته است. هرچه این کران به هزینه واقعی جواب بهینه نزدیک‌تر باشد، تعداد بیشتری از شاخه‌های غیرامیدبخش در همان مراحل ابتدایی حذف خواهند شد.

در این پروژه، برای محاسبه کران پایین از ایده الگوریتم کراسکال استفاده شده است.

فرض کنید در یکی از گره‌های درخت جستجو قرار داریم. در این وضعیت، تعدادی از کابل‌ها قبلاً انتخاب شده‌اند و ساختار DSU وضعیت فعلی مؤلفه‌های همبند را نگهداری می‌کند. همچنین مقدار هزینه ساخت فعلی برابر با

$$C_{current}$$

است.

ابتدا یک نسخه موقت از ساختار DSU ساخته می‌شود تا تغییرات ایجادشده در هنگام محاسبه کران پایین بر وضعیت واقعی گره تأثیری نداشته باشد.

سپس کابل‌های باقی‌مانده که از قبل بر اساس هزینه ساخت مرتب شده‌اند، دقیقاً مشابه الگوریتم کراسکال بررسی می‌شوند. هر کابلی که موجب تشکیل چرخه نشود، به صورت موقت انتخاب شده و هزینه ساخت آن به مقدار فعلی افزوده می‌شود. این فرآیند تا زمانی ادامه پیدا می‌کند که تعداد یال‌های لازم برای تکمیل درخت پوشا تأمین شود.

اگر تعداد یال‌های موردنیاز برای تکمیل درخت برابر باشد با

$$need = (n - 1) - EdgeCount,$$

کران پایین هزینه ساخت به صورت زیر محاسبه می‌شود

$$LB = C_{current} + \sum_{i=1}^{need} w_i,$$

که در آن، مجموع فوق مربوط به ارزان‌ترین کابل‌های قابل انتخاب مطابق روند الگوریتم کراسکال است.

در صورتی که حتی با استفاده از تمام کابل‌های باقی‌مانده نیز امکان تکمیل درخت پوشا وجود نداشته باشد، مقدار

$$LB = \infty$$

در نظر گرفته می‌شود و آن شاخه مستقیماً حذف خواهد شد.

این روش در عمل همان ادامه دادن اجرای الگوریتم کراسکال از وضعیت فعلی گره است؛ با این تفاوت که انتخاب‌ها تنها برای محاسبه یک کران خوش‌بینانه انجام می‌شوند و هیچ تغییری در وضعیت واقعی الگوریتم ایجاد نمی‌کنند.

۹.۱ صحت تابع کران پایین

برای محاسبه کران پایین، تنها هزینه ساخت در نظر گرفته می‌شود و محدودیت بودجه نگهداری عمداً نادیده گرفته می‌شود. بنابراین، فضای انتخاب در محاسبه کران پایین بزرگ‌تر از فضای جواب‌های مجاز مسئله اصلی خواهد بود.

در نتیجه، مقدار حاصل هیچ‌گاه از هزینه ساخت بهترین جواب قابل دستیابی از آن گره بیشتر نخواهد بود.

به عبارت دیگر، اگر هزینه هر جواب قابل قبول از این گره را با $Cost$ نمایش دهیم، خواهیم داشت

$$LB \leq Cost.$$

از آنجا که جواب بهینه نیز یکی از جواب‌های قابل قبول است، نتیجه می‌شود

$$LB \leq OPT.$$

بنابراین مقدار محاسبه شده همواره یک کران پایین معتبر برای هزینه ساخت جواب بهینه محسوب می شود.

در نتیجه، هرگاه

$$LB \geq BestCost$$

باشد، هیچ جوابی در آن شاخه قادر نخواهد بود هزینه ای کمتر از بهترین جواب فعلی تولید کند؛ بنابراین حذف آن شاخه کاملاً ایمن بوده و هیچ جواب بهینه ای از بین نخواهد رفت.

۱۰.۱ شرایط هرس

در ابتدای بررسی هر گره، مجموعه ای از شروط هرس کنترل می شوند. در صورتی که هر یک از این شرایط برقرار باشد، ادامه بررسی آن شاخه متوقف شده و الگوریتم مستقیماً به شاخه بعدی می رود.

در پیاده سازی چهار شرط هرس در نظر گرفته شده است.

۱. اگر مجموع هزینه نگهداری کابل های انتخاب شده از بودجه تعیین شده بیشتر شود،

$$Maintenance > B,$$

ادامه آن شاخه بی معنا خواهد بود؛ زیرا اضافه شدن کابل های جدید تنها هزینه نگهداری را افزایش می دهد و امکان بازگشت به یک جواب مجاز وجود ندارد.

۲. اگر تعداد کابل های باقی مانده برای تشکیل یک درخت پوشا کافی نباشد،

$$RemainingEdges < (n - 1) - EdgeCount,$$

آن شاخه حذف می شود؛ زیرا تکمیل درخت از نظر ساختاری غیرممکن است.

۳. اگر هزینه ساخت فعلی برابر یا بیشتر از بهترین جواب به دست آمده باشد،

$$BuildCost \geq BestCost,$$

نیازی به ادامه بررسی آن شاخه وجود ندارد؛ زیرا هزینه ساخت در ادامه مسیر تنها افزایش خواهد یافت و هیچ فرزندی نمی تواند هزینه ای کمتر از مقدار فعلی داشته باشد.

۴. در نهایت، اگر

$$LowerBound \geq BestCost,$$

حتی خوش‌بینانه‌ترین ادامه ممکن نیز قادر به تولید جوابی بهتر از بهترین جواب فعلی نخواهد بود؛ بنابراین آن شاخه بدون بررسی بیشتر حذف می‌شود.

ترکیب این چهار شرط باعث می‌شود بخش بزرگی از فضای جستجو حذف شده و تنها شاخه‌هایی بررسی شوند که همچنان احتمال رسیدن به جواب بهینه را دارند.

۱۱.۱ تشخیص جواب نهایی

در هر گره از درخت جستجو، پس از اعمال شروط هرس، بررسی می‌شود که آیا یک درخت پوشای معتبر تشکیل شده است یا خیر.

در پیاده‌سازی، زمانی یک جواب معتبر به دست آمده است که شرایط زیر به طور هم‌زمان برقرار باشند:

$$EdgeCount = n - 1$$

و

$$MaintenanceCost \leq B.$$

شرط اول تضمین می‌کند که تعداد یال‌های انتخاب‌شده برای تشکیل یک درخت پوشا کافی است و شرط دوم نیز رعایت محدودیت بودجه نگهداری را تضمین می‌کند.

در صورتی که هر دو شرط برقرار باشند، هزینه ساخت جواب فعلی با بهترین جواب یافت‌شده تاکنون مقایسه می‌شود. اگر هزینه ساخت کمتر باشد، مقدار بهترین هزینه و همچنین مجموعه کابل‌های انتخاب‌شده به‌روزرسانی می‌شوند.

اگر پس از پایان جستجو هیچ جواب معتبری پیدا نشود، مسئله فاقد پاسخ بوده و عبارت

NO

چاپ می‌شود. در غیر این صورت، بهترین جواب به‌دست‌آمده به عنوان پاسخ نهایی گزارش خواهد شد.

۱۲.۱ پیچیدگی زمانی

در بدترین حالت، الگوریتم Branch and Bound ممکن است تمام زیرمجموعه‌های ممکن از کابل‌های جدید را بررسی کند. بنابراین تعداد گره‌های درخت جستجو در بدترین حالت برابر است با

$$O(2^e),$$

که در آن e تعداد کابل‌های قابل بررسی است.

در هر گره، برای محاسبه تابع کران پایین، ادامه‌ای خوش‌بینانه از الگوریتم کراسکال اجرا می‌شود. این محاسبه با استفاده از ساختار DSU انجام شده و در بدترین حالت تقریباً

$$O(e \alpha(n))$$

زمان نیاز دارد که در آن $\alpha(n)$ تابع معکوس آکرمن است و در عمل مقدار بسیار کوچکی دارد.

در نتیجه، مرتبه زمانی بدترین حالت الگوریتم برابر است با

$$O(2^e \times e \alpha(n)).$$

البته این مرتبه تنها یک کران نظری است. در عمل، به دلیل استفاده از تابع کران پایین مبتنی بر الگوریتم کراسکال، ساختار داده DSU و شروط مختلف هرس، بخش بزرگی از فضای جستجو حذف شده و تعداد حالت‌های بررسی شده به طور قابل توجهی کاهش می‌یابد.

۱۳.۱ جمع‌بندی

در این فاز، مسئله بازسازی شبکه به صورت یک مسئله Constrained Minimum Spanning Tree مدل‌سازی شد که در آن علاوه بر کمینه‌سازی هزینه ساخت، محدودیت بودجه نگهداری نیز باید رعایت شود.

وجود این محدودیت سبب می‌شود خاصیت انتخاب حریصانه از بین ورود و الگوریتم‌های کلاسیک درخت پوشای کمینه مانند کراسکال و پریم دیگر قادر به تضمین جواب بهینه نباشند. به همین دلیل، برای حل مسئله از الگوریتم Branch and Bound استفاده شد.

در این روش، فضای تمام جواب‌های ممکن به صورت سیستماتیک پیمایش می‌شود و با استفاده از یک تابع کران

پایین مبتنی بر ادامه خوش بینانه الگوریتم کراسکال، شاخه‌هایی که امکان رسیدن به جوابی بهتر از بهترین جواب فعلی را ندارند حذف می‌شوند. همچنین استفاده از ساختار داده DSU موجب می‌شود بررسی تشکیل چرخه و محاسبه کران پایین با هزینه بسیار اندکی انجام شود.

اگرچه الگوریتم در بدترین حالت دارای پیچیدگی نمایی است، اما استفاده هم‌زمان از مرتب‌سازی اولیه کابل‌ها، کران پایین مناسب، ساختار DSU و شروط هرس باعث کاهش چشمگیر فضای جستجو در عمل می‌شود. در نتیجه، الگوریتم ضمن تضمین یافتن جواب بهینه، از نظر عملی نیز کارایی قابل قبولی برای ابعاد مسئله ارائه می‌دهد.